

**Sistemi Operativi**  
**Unità 2: Utilizzo di Linux**  
**Ambienti Linux**

Martino Trevisan  
Università di Trieste  
Dipartimento di Ingegneria e Architettura

## Argomenti

1. Distribuzioni di Linux
2. Alternative per usare Linux

# Distribuzioni di Linux

## Distribuzioni di Linux

Con **Linux** si intende il Kernel UNIX-like creato da Linus Torvald.

Con **GNU/Linux** si intende una famiglia di sistemi operativi basati su Kernel Linux

Ci sono più di 100 SO che sono della famiglia **GNU/Linux**.  
Condividono:

- Kernel Linux
- Programmi e utility di base di GNU per gestione di file, processi, rete

## Distribuzioni di Linux

Le famiglie principali OS Linux sono:

**Ubuntu:** attualmente il più diffuso.

- Basato su un'altra distribuzione chiamata **Debian**
  - Debian è il punto di partenza per tanti altri OS Linux
  - Debian contiene solo software libero, Ubuntu no
- Ha lo scopo di offrire un SO completo e facile da usare per PC (e per server)
- Ne derivano altre distribuzioni che si differenziano per il software che gestisce l'ambiente grafico (desktop, finestre)

## Distribuzioni di Linux

**Red Hat Enterprise Linux e CentOS:** versioni professionali di Linux, per il mercato aziendale

- Particolare attenzione a stabilità e sicurezza
- Mantenate dall'azienda Red Hat, che offre supporto a pagamento
- RHEL è la versione con supporto commerciale. CentOS è la versione *consumer*
- L'OS **Fedora** è della stessa famiglia, è adotta funzionalità più innovative, sebbene meno stabili

## Distribuzioni di Linux

**Arch Linux:** distribuzione leggera, adatta a sistemi minimali e con poche risorse

- Non prevede un ambiente Desktop di default
- Utilizza la filosofia KISS (Keep It Simple, Stupid)

**openSUSE Linux:** sviluppata da volontari. Nei primi anni 2000 era molto diffusa

**Linux Mint:** basata su Ubuntu. Ha avuto momenti di celebrità nei primi anni 2010

# Distribuzioni di Linux

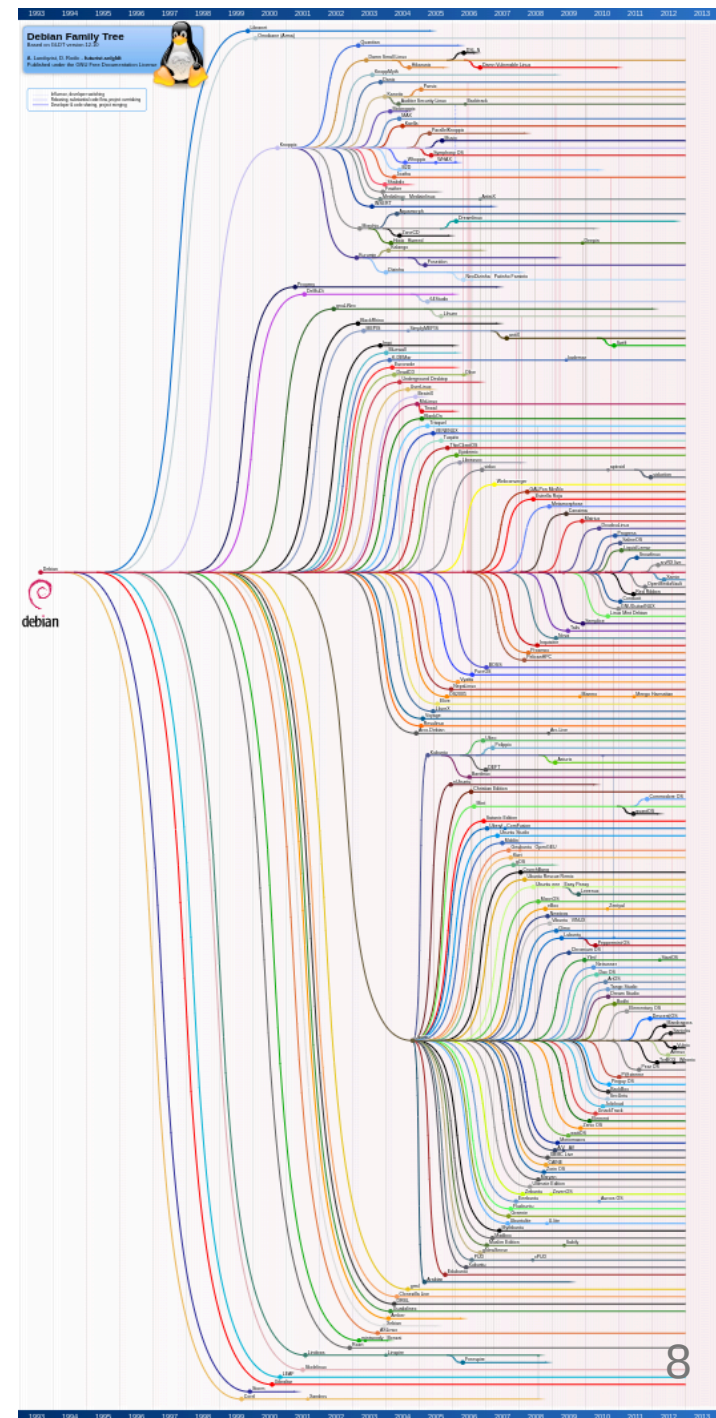
Impossibile enumerare tutte le distribuzioni.

Molte nascono e muoiono nel giro di pochi anni

In questo corso utilizzeremo

## Ubuntu

- Diffuso
- Semplice
- Generico
  - Ha versione per PC e per server



# Alternative per usare Linux

## Alternative per usare Linux

Per utilizzare un sistema Linux, ci sono varie alternative a seconda che:

- Si abbia in PC o un MAC
- Si abbia tanto o poco spazio su disco
- Si sia più o meno esperti nell'utilizzo del computer

## Alternative per usare Linux

**Installazione Nativa:** si installa un SO Linux su un PC.

- Necessario scaricare l'immagine dal sito in un SO Linux (e.g., Ubuntu)
- Il PC viene formattato e il SO è installato nativamente
- Si può mantenere Windows (o Mac OS) usando il **Dual Boot**
  - L'hard disk è partizionato in due drive logici, uno con Linux, uno con Windows
- Operazione non facilissima, e potenzialmente distruttiva

## Alternative per usare Linux

### Linux da USB "Live":

Ogni distribuzione di Linux può essere usata Live:

- Si crea una chiavetta USB *bootable*
- Si inserisce nel PC e lo si forza a fare *boot* da chiavetta
- Linux gira nativamente come se fosse installato
- Su Windows si può usare il software Rufus

<https://rufus.ie/en/>

- **Nota:** a meno che non lo si configuri esplicitamente, la chiavetta non è **persistente**. A ogni riavvio si perdono tutti i file modificati

## Alternative per usare Linux

**Macchina Virtuale:** utilizzando un software chiamato **virtualizzatore** è possibile creare un PC virtuale.

- E' a tutti gli effetti in PC completo di tutte le funzionalità
  - Ha una CPU, memoria e disco virtuali
- Che gira all'interno di un'applicazione
  - Non danneggia nè impatta il SO nativo del PC
- Tanti software per virtualizzazione
  - **VirtualBox** (consigliato)
  - **VMWare**
  - **QEMU**

## Alternative per usare Linux

- Passi necessari:
  - Installare il virtualizzatore
  - Creare una nuova macchina virtuale
    - Specificare la quantità di risorse (CPU, memoria, disco) da allocare alla macchina virtuale
  - Installare il SO Linux preferito
  - Configurarlo con i software desiderati, se necessario
- Questa è l'**opzione consigliata**:
  - Facile, stesse potenzialità di avere Linux installato nativamente
  - Il PC deve essere abbastanza potente:
    - Almeno 8 core, 8GB di RAM e 20GB (di spazio libero) su Hard Disk

## Alternative per usare Linux

**Cygwin:** è un software da installare su Windows

E' un layer di compatibilità POSIX che permette di usare programmi POSIX su sistemi Windows

- Mappa le system call POSIX su quelle di Windows.
- Include i tool GNU base per gestione di file, compilazione
- Necessario compilare i programmi usando Cygwin

Facile da installare:

- Si installa come un normale programma
- Sito Web: [www.cygwin.com](http://www.cygwin.com)

## Alternative per usare Linux

**Windows Subsystem for Linux (WSL):** permette di eseguire programmi Linux su Windows (tramite una VM).

- Sviluppato direttamente da Microsoft
- A partire da Windows 10
- Permette di eseguire eseguibili Linux **senza ricompilare**
- Si può installare tramite command line di Windows
- Dopodichè è possibile installare pacchetti di software Linux
  - Ad esempio si può installare l'applicazione "Ubuntu" tramite software center.
  - Nota: l'applicazione "Ubuntu" non è un vero SO. E' solo un pacchetto che contiene i software di base di Ubuntu e una shell

## Alternative per usare Linux

### Terminale via browser:

- Tante opzioni online
  - Cercare su Google: *linux box online*
- Una è: <https://linuxcontainers.org/incus/try-it/>
  - Non persistente
  - Limitata a 30 minuti
  - Va bene per provare i comandi Linux *al volo*

## Domande

Ubuntu é un SO che utilizza il Kernel:

- Linux
- UNIX
- POSIX

Red Hat é un:

- Kernel
- SO
- Uno standard

**Sistemi Operativi**  
**Unità 2: Utilizzo di Linux**  
**Comandi di Linux**

Martino Trevisan  
Università di Trieste  
Dipartimento di Ingegneria e Architettura

## Argomenti

1. Sessione Shell
2. Comandi di base
3. File System
4. Utenti e permessi
5. Processi e programmi
6. Altri comandi
7. Esercizi

# Sessione Shell

## Sessione Shell

Per fare login, è necessario inserire le proprie credenziali sul terminale:

```
login as: <username>  
password: <password>
```

Per fare logout: `CTRL+D`, `exit` o `shutdown` (solo superuser)

## Sessione Shell

E' possibile usare un terminale *remoto* utilizzando SSH.

Su un'altra macchina collegata in rete, digitare su terminale:

```
ssh <username>@<indirizzo IP della macchina>
```

Si utilizza il protocollo Secure Shell, che trasmette in maniera cifrata i comandi e il loro output tramite la rete.

# Comandi di base

## Comandi di base

Vengono digitati sul terminale. Avviano il corrispondente eseguibile

Alcuni ammettono **argomenti**, gli oggetti su cui il comando deve agire.

- Specificati dopo il nome del comando

Alcuni ammettono **opzioni** che specificano comportamenti particolari

- Iniziano per `-` seguite da una singola lettera
- Oppure per `--` seguite da una stringa

## Comandi di base

### Formato:

```
comando [opzioni] [argomenti]
```

**Esempio:** stampa il contenuto di `file.txt`

```
cat file.txt
```

**Esempio:** lista il contenuto della cartella `dir`, includendo anche i file nascosti (che iniziano per `.`):

```
ls -a dir
```

```
ls --all dir
```

## Comandi di base

E' possibile avere più comandi con una sola riga, separandoli con `;`.

```
comando1 ; comando2; ...
```

### Altri comportamenti:

- I comandi possono essere concatenati tramite il carattere `|`.
- Si può redirezionare l'output di un comando su file tramite il carattere `>`.
- Analizzato in dettaglio più avanti

## Comandi di base

**Manuale in linea:** i comandi sono documentati

```
man <comando>
```

Restituisce la pagina di manuale del `<comando>` .

Comandi simili:

- `apropos` : ricerca in tutti i manuali dei comandi
- `whereis` : trova il binario, il sorgente e il manuale di un comando

## Comandi di base

### Altri comandi di base:

- `date` : visualizza la data
- `who` : mostra gli utenti attualmente collegati.
- `uptime` : tempo di vita di un sistema, numero di utenti collegati, carico del sistema negli ultimi 1, 5, 15 minuti
- `hostname` : nome della macchina

# **File System**

## File System

Il file system su Linux è gerarchico.

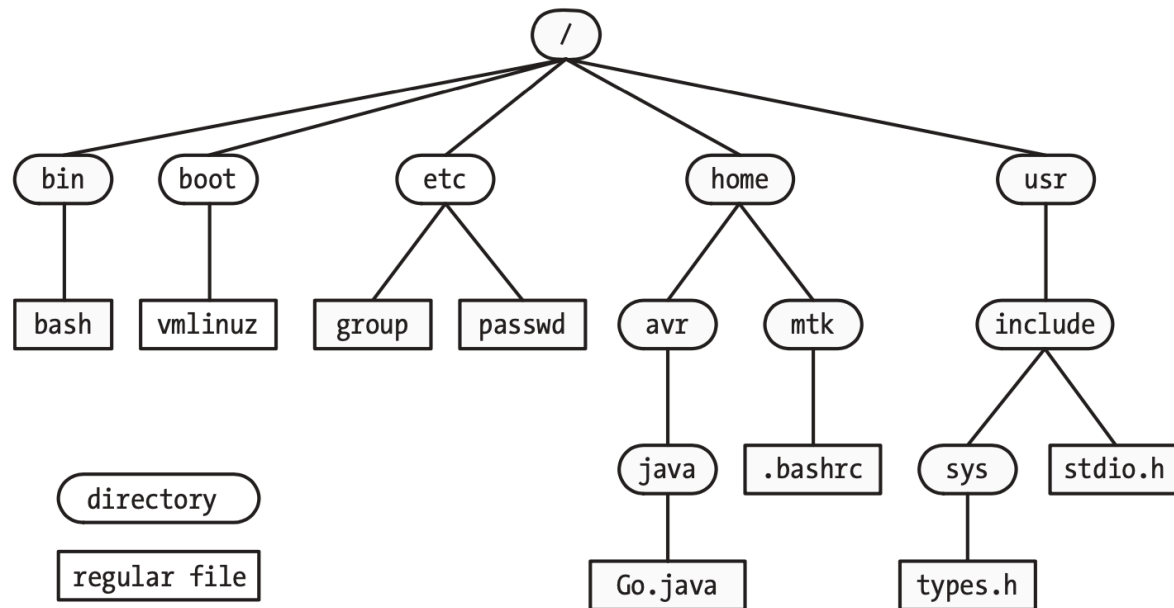
- Organizzato in directory annidate l'una dentro l'altra
- La directory radice è `/`
- Tutte le cartelle del sistema sono contenute nella directory radice.

Esempio: la home degli utenti si trova in

`/home/nomeutente`

## File System

Esempio di albero (parziale) delle cartelle di un sistema Linux



## File System

Un **path** identifica un file o una cartella.

Un terminale (e i comandi che vi vengono lanciati) sono sempre **posizionati** su una cartella, la **Working Directory**.

- Ci muove da una cartella all'altra col comando `cd`  
`<path>`
- Un path può essere:
  - Assoluto: inizia con `/` e indica un path completo a partire dalla radice
  - Relativo: **non** inizia con `/` e indica un path a partire dalla Working Directory
- Ci si può riferire directory corrente con `.`
- La directory padre di quella corrente è indicata con `..`

## File System

**Esempio:** si consideri il seguente albero di cartelle

```
/
└─ tmp
    └─ directory
        └─ file.txt
```

Col comando `cd /tmp` si posiziona il terminale in `/tmp`.

A questo punto:

- `directory` identifica la directory `/tmp/directory`
- `.` identifica la directory `/tmp`
- `..` identifica la directory `/`
- `directory/../../` rappresenta la directory `/`

## File System

Su tutti i sistemi Linux, il file system è organizzato con le seguenti cartelle di sistema.

- `/` : radice
- `/bin` : file eseguibili del sistema – ls, pwd, cp, mv ....
- `/boot` : file necessari per l'avvio del sistema, boot loader  
...
- `/dev` : file speciali che descrivono i dispositivi – dischi, scheda audio, porte seriali ...
- `/etc` : file eseguibili, script, inizializzazione, configurazione sistema, file password, ...

## File System

- `/home` : directory delle home directory degli utenti
- `/lib` : librerie di sistema
- `/lost+found` : contiene i file danneggiati
- `/mnt` : punto di montaggio file system (mount point)
- `/proc` : file system virtuale che contiene informazioni sui programmi in esecuzione
- `/sys` : programmi di sistema
- `/tmp` : direttorio temporaneo

## File System

- `/usr` : file relativi alle applicazioni installate
- `/usr/bin` : file binari disponibili agli utenti
- `/usr/lib` : file binari disponibili agli utenti
- `/usr/include` : header file libreria standard C
- `/var` : file di sistema che variano con frequenza elevata

## File System

**Comando** `ls [opzioni] [dir]` : lista il contenuto della directory. Opzioni principali:

- `-1` : stampa su una colonna
- `-l` : formato lungo
- `-n` : come `-l` ma visualizza gli ID al posto del nome del proprietario e del gruppo
- `-t` : ordina per data
- `-s` : mostra la dimensione dei file in blocchi
- `-a` : mostra tutti i file compresi `.` e `..`
- `-R` : elenca il contenuto in modo ricorsivo

## File System

**Esempio:** differenti forme di `ls`

```
$ ls
compile.txt  style.css  u1-introduzione  u2-linux
```

```
$ ls -l
total 16
-rw-rw-r-- 1 martino martino 102 set 30 14:16 compile.txt
-rw-rw-r-- 1 martino martino 199 set 30 15:27 style.css
drwxrwxr-x 3 martino martino 4096 ott  1 18:33 u1-introduzione
drwxrwxr-x 3 martino martino 4096 ott  4 10:20 u2-linux
```

## File System

**Comando** `rm [-rfi] [filename]` : rimuove il/i file selezionati. Opzioni principali:

- `-r` : rimozione ricorsiva del contenuto delle directories.
- `-f` : rimozione di tutti i file (anche protetti in scrittura) senza avvisare.
- `-i` : con questa opzione `rm` chiede conferma

**Esempio:** cancella tutti i file in `cartella`

```
rm cartella/*
```

**Nota:** con `*` si intendono tutti i file dentro una cartella

## File System

**Comando** `cd <dir>` : cambia directory.

**Comando** `mkdir <dir>` : crea sub-directory.

**Comando** `rmdir <dir>` : rimuove sub-directory, solo se vuota. Altrimenti fallisce.

**Comando** `cp <file1> <file2>` e `mv <file1> <file2>` : copia/sposta file o cartelle. Opzioni principali

- `-f` : effettua le operazioni senza chiederne conferma
- `-i` : chiede conferma nel caso che la copia sovrascriva il file di destinazione
- `-r` : ricorsivo (solo `cp` ). Copia la directory e tutti i suoi file, incluso le sottodirectory ed i loro file

## File System

**Comando** `ln [-s] <sorgente> <destinazione>` : crea un **link**. In Linux esistono due tipi di link:

- **Soft Link**: è un semplice rimando a un altro path. Se il path destinazione non esiste o viene spostato, il link semplicemente non funziona. Si usa l'opzione `-s`.
  - **Flessibile**: può linkare a un altro file system o a una cartella
- **Hard Link**: associa un secondo path al contenuto del file. Se il primo file viene spostato, il link rimane valido e funzionante. E' l'opzione di default
  - **Robusto**: non può mai essere invalido. Non si può usare tra dischi diversi, nè per linkare cartelle

## File System

**Comando** `find [path] [-n nome] [-print]` : ricerca ricorsiva di directories

**Esempio:** cerca i file che finiscono per `.txt` nella directory `/tmp` :

```
find /tmp -name *.txt
```

E' possibile filtrare su varie **proprietà** dei file o cartelle:

- Tempo di creazione/modifica
- Utente o gruppo proprietario
- Grandezza

**Nota:** Non effettua ricerca nel contenuto del file

## File System

**Comando** `cat <file>` : stampa il contenuto di un file

**Comando** `touch <file>` : crea il file se non esiste

**Esempio:** creare un file `a.txt` , aprirlo con un editor e scrivervi dentro `ciao` , poi stampare il file

```
$ touch a.txt
... modificare con editor
$ cat a.txt
ciao
```

## File System

**Comando** `less <file>` : apre il file in un visualizzatore interno alla shell dove si può scorrere in entrambe le direzioni

Esistono svariati altri comandi per visualizzare il contenuto di un file.

- Comandi per stampare file binari ( `hexdump` )
- Comandi per stampare le prime ( `head` ) o le ultime righe ( `tail` ) di un file
- Editor avanzati utilizzabili dentro la shell.
  - `nano` il più semplice
  - Ne esistono molti. Sono in competizione `emacs` e `vi`
    - Vedere: [https://it.wikipedia.org/wiki/Guerra\\_degli\\_editor](https://it.wikipedia.org/wiki/Guerra_degli_editor)

# Utenti e permessi

## Utenti e permessi

Un dispositivo con OS Linux può avere più **utenti**.

- Essi possono fare login su una shell o un terminale remoto
- Ogni utente ha la sua **Home Directory** in

`/home/<utente>`

Serve per permettere all'utente di immagazzinare file personali come documenti, immagini, programmi.

## Utenti e permessi

Un utente può essere assegnato a uno o più **gruppi**.

- Ogni utente deve avere almeno un gruppo, detto **gruppo primario**
- Un utente può essere assegnato a zero o più **gruppo secondari**

Meccanismo utente-gruppi utilizzato per gestire l'accesso a file e risorse.

L'utente **root** esiste sempre ed ha massimi privilegi

## Utenti e permessi

**Gestione:** Comandi per creare o rimuovere utenti e gruppi:

`useradd` , `groupadd` , `userdel` , `groupdel`

- Su molti OS Linux, esistono dei comandi più facili:

`adduser` , `addgroup` , `deluser` , `delgroup`

### Altri comandi:

- `groups` : stampa i gruppi ai quali appartiene l'utente corrente
- `whoami` : stampa l'utente corrente
- `su <utente>` : cambia utente (chiede password)
- `sudo <comando>` : esegue il comando come utente `root` , dopo aver chiesto la password

## Utenti e permessi

I file e le cartelle hanno tre tipi di **permessi**:

- Permesso di **Lettura**: Per i file, accedere a contenuto. Per cartelle, listare i file.
- Permesso di **Scrittura**: Per i file, modificare il contenuto. Per le cartelle, creare file o cartelle in essa.
- Permesso di **Esecuzione/Attraversamento**:
  - Per i **file**, esiste il permesso di **esecuzione**. Necessario per eseguire programmi.
  - Per le **cartelle**, esiste il permesso di **attraversamento**. Necessario per accedere a sotto cartelle.

## Utenti e permessi

I file e le cartelle hanno un **utente proprietario** e un **gruppo proprietario**.

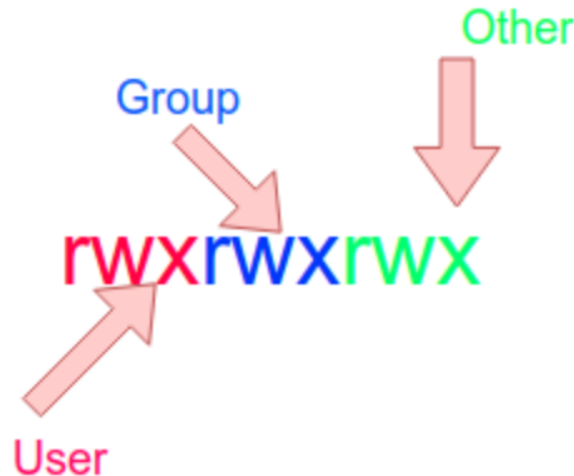
I permessi su file sono gestibili separatamente per:

- Utente proprietario del file
- Utenti del gruppo proprietario del file
- Tutti gli altri utenti

## Utenti e permessi

In totale ogni file o cartella ha  $3 \times 3$  permessi

- Ogni permesso può essere attivo o no



### Esempio:

```
$ ls -l
-rw-rw-r-- 1 martino docenti 102 set 30 14:16 compile.txt
-rw-rw-r-- 1 martino docenti 199 set 30 15:27 style.css
```

## Utenti e permessi

**Modifica dei permessi di un file:** si usa il comando `chmod`

`[-r] <permessi> <file>`

I permessi possono essere indicati con varie sintassi:

- **Assoluto**, con tre cifre ottali, che rappresentano rispettivamente i permessi a utente, gruppo e altri. Ogni cifra ha 3 bit e rappresenta permessi di lettura, scrittura ed esecuzione/attraversamento.

Esempio: `chmod 750 file.txt` dà permessi totali a utente ( $7_8 = 111_2$ ), lettura/esecuzione al gruppo ( $5_8 = 101_2$ ) e niente agli altri ( $0_8 = 000_2$ )

## Utenti e permessi

- **Mirato:** Modifica permessi esistenti tramite una stringa composta di 3 parti:
  - Quali utenti: `u` (user), `g` (gruppo), `o` (other)
  - Che operazione: `+` (aggiungi), `-` (rimuovi)
  - Quale permesso: `r` (lettura), `w` (scrittura), `x` (esecuzione/attraversamento)Esempio: `chmod g+w file.txt` dà permessi in scrittura agli utenti del gruppo proprietario del file
- `-r` applica il comando ricorsivamente a file e cartelle contenute
- **Chi può modificare i permessi:** Utente proprietario e utente `root`

## Utenti e permessi

**Esempio:** usi di `chmod`

- `chmod 600 file.txt` : l'utente può leggere e scrivere. Il gruppo e gli altri niente.
- `chmod 640 file.txt` : l'utente può leggere e scrivere. Il gruppo può leggere. Gli altri niente
- `chmod u+x file.txt` : Aggiungi i permessi di esecuzione all'utente
- `chmod go+w file.txt` : Aggiungi i permessi di scrittura al gruppo e a gli altri

## Utenti e permessi

### Modifica di proprietario e gruppo di file o cartella

- `chown utente file` : modifica utente proprietario
- `chgrp gruppo file` :
- `chown utente:gruppo file` : modifica contemporaneamente entrambi
- **Note:**
  - Posso assegnare un file solo a un gruppo che possiedo
  - Sulla maggior parte degli OS, solo `root` può cambiare utente proprietario
  - Opzione `-r` : applica il comando ricorsivamente a cartelle e file contenuti

## Utenti e permessi

### Esempio:

si considerino le seguenti informazioni sul file `my-program`:

```
-rwxr-xr-- 1 martino docenti 102 set 30 14:16 my-program
```

- L'utente `luca` del gruppo `docenti` può eseguire `my-program` ?

**SI:** il gruppo ha permessi `r-x`, quindi `luca` può eseguire `my-program`

- L'utente `marco` del gruppo `studenti` può eseguire `my-program` ?

**NO:** gli altri hanno permessi `r--`, quindi `marco` non può eseguire `my-program`

## Utenti e permessi

### Esempio:

si considerino le seguenti informazioni sulla cartella `data` :

```
dr-xr--r-- 3 martino docenti 4096 ott  1 18:33 data/
```

- L'utente `luca` del gruppo `docenti` può listare i file?

**SI:** il gruppo ha permessi `r--`

- L'utente `luca` del gruppo `docenti` può accedere alla cartelle dentro `data` ?

**NO:** il gruppo ha permessi `r--` . Servirebbe `r-x`

- L'utente `martino` del gruppo `docenti` può creare file in `data` ?

**NO:** l'utente `martino` ha permessi `r-x` . Servirebbe `rwX`

# Processi e programmi

## Processi e programmi

Un processo è un programma in esecuzione.

In Linux, ogni processo è identificato da un'identificatore detto **PID**.

Il PID si usa per effettuare operazioni sul processo.

**kill <PID>** : termina il processo

**top** : mostra in maniera interattiva i processi in esecuzione.

Simile a un Task Manager via Shell

## Processi e programmi

**ps [opzioni]** : mostra informazioni sui processi attivi.

- **a** : informazioni su tutti i processi (non solo generati dalla sessione shell corrente)
- **x** : mostra anche i processi in background
- **f** : stampa i processi in modo che se ne veda il rapporto padre-figli
- **u** : stampa più informazioni

**Esempio:** `ps fax`

**Nota:** `ps` è tra i pochi programmi in cui le opzioni non vanno iniziate con `-`. Ciò è un relitto delle primissime versioni di Unix in cui le opzioni non avevano il `-`.

## Altri comandi

**lsusb** : lista i dispositivi usb

**lspci** : lista i dispositivi su bus pci

**lsblk** : lista i dischi

**ifconfig** : lista le interfacce di rete

**pwd** : stampa la directory corrente

**free** : mostra quanta memoria RAM libera ed occupata ha il sistema

## Altri comandi

**df [-htv]** : visualizza informazioni sui file system del sistema.

- **-t** : nr totale di blocchi e i-node liberi
- **-v** : percentuale di blocchi e i-node
- **-h** : stampa in GB/MB anziché in numero di byte

| Filesystem | 1K-blocks | Used      | Available | Use% | Mounted on     |
|------------|-----------|-----------|-----------|------|----------------|
| udev       | 3923948   | 4         | 3923944   | 1%   | /dev           |
| tmpfs      | 787220    | 1552      | 785668    | 1%   | /run           |
| /dev/sda6  | 425085288 | 259786508 | 143682652 | 65%  | /              |
| none       | 4         | 0         | 4         | 0%   | /sys/fs/cgroup |
| none       | 5120      | 0         | 5120      | 0%   | /run/lock      |
| none       | 3936080   | 436       | 3935644   | 1%   | /run/shm       |
| none       | 102400    | 44        | 102356    | 1%   | /run/user      |

# Esercizi

## Esercizi

1. Stampare il contenuto del file `/etc/hosts`
2. Posizionarsi nella cartella `/tmp` e listare il contenuto della cartella `/home` usando un path relativo e uno assoluto
3. Usare l'editor nano per creare un file `file.txt` in una qualsivoglia cartella. Creare una link simbolico del file e cancellare l'originale. Cosa succede se si prova a stampare il contenuto del link? Ripetere con Hard Link
4. Creare una cartella e due file in essa. Cancellare la cartella con un unico comando.

## Esercizi

5. Creare un nuovo gruppo `studenti` e un utente `studente` assegnato a tale gruppo.

**Nota:** usare le opzioni `-m` `-g <group>` del comando `useradd`

E' necessario usare `sudo` ?

Un utente normale può listare i file nella home della home directory di `studente` ?

Modificare i permessi della home di `utente` affinché tutti possano leggere, scrivere ed eseguire

**Sistemi Operativi**  
**Unità 2: Utilizzo di Linux**  
**Programmi in Bash**

Martino Trevisan  
Università di Trieste  
Dipartimento di Ingegneria e Architettura

## **Argomenti**

1. Script Bash
2. Variabili
3. Strutture di controllo
4. Esercizi

# Script Bash

## Script Bash

### Definizione

**Bash** è il software shell di default in GNU/Linux

Permette di:

- Eseguire comandi (già visto)
- Definire variabili
- Controllare il flusso (`if`, `while`, ecc...)

E' un linguaggio completo di tutti i costrutti.

Ha una sintassi particolare e problematica

Una lista di comandi può essere racchiusa in un file detto **script**.

# Script Bash

## Esempio

Uno script di esempio nel file `script.sh` :

```
#!/bin/bash  
# primo esempio di script  
echo $RANDOM
```

`#!/bin/bash` Indica che il file è uno script bash

`# primo esempio di script` E' un commento

`echo $RANDOM` Stampa il contenuto della variabile `$RANDOM`

Per eseguire lo script.

```
./script.sh
```

Il file deve avere permessi di lettura ed esecuzione

## Script Bash

### Valore di ritorno

Uno script è una lista di comandi che vengono eseguiti più delle strutture di controllo.

```
ls # Lista i file  
./myprog # Avvia il programma myprog
```

Ogni processo in Linux/POSIX deve fornire un **Valore di Ritorno** al chiamante (il **Processo Padre**), ovvero:

- La shell
- Uno script bash
- Un qualsiasi altro processo

## Script Bash

### Valore di ritorno

Il valore di ritorno è un numero intero

- Usato dal Processo Padre per vedere se c'è stato errore
- Per convenzione **0** se successo, **≠ 0** in caso di errore

In uno script bash, si accede al Valore di Ritorno dell'ultimo comando tramite `$?`

## Script Bash

### Parametri di riga di comando

Si possono usare in uno script. Si ottengono con:

- `$1` , `$2` , ...: contenuti dei parametri
- `$0` : nome dello script
- `$#` : numero di argomenti

**Esempio:** il nome dello script è `script.sh` e viene eseguito come `./script.sh ciao`.

```
#!/bin/bash
echo $0 # Stampa "./script.sh"
echo $1 # Stampa "ciao"
```

# Variabili

## Variabili

### Nomi

Combinazione illimitata di lettere, numeri e underscore.

Non possono cominciare con numeri e sono CASE sensitive.

## Lettura di variabili da terminale

Istruzione `read nomevar`

```
read nome
```

## Stampa su schermo

Si utilizza `echo` .

```
echo "Testo su schermo"
```

## Variabili

### Utilizzo di variabili

Si accede con `$nomevar` .

```
read x    # Legge X da terminale
y=$x      # Assegna a y il valore di x
echo $y   # Stampa quanto letto
```

I tipi principali sono:

- **Stringhe:** `a="testo"`
- **Interi:** `a=47`

**Nota:** non inserire spazi prima e dopo `=` durante assegnazione

## Variabili

## Quoting

Le stringhe vanno racchiuse tra `"` o tra `'`.

Il carattere `\` è di **escape**. Permette di usare nella stringa il carattere di quoting

### Esempio:

`a="ciao"` indica la stringa `ciao`

`a='ciao a tutti'` indica la stringa `ciao a tutti`

`a="ha detto: \"ciao\""` indica la stringa `ha detto: "ciao"`

## Variabili

## Quoting

### Quoting:

Le stringhe definite con `"` possono contenere delle variabili che vengono valuate.

```
a="test"  
b="this is a $a"  
c='this is a $a'
```

La variabile `b` contiene `this is a test`

La variabile `c` contiene `this is a $a`

## Variabili

**Esempio:** Si crei uno script Bash che chiede all'utente di inserire due stringhe e stamparle.

```
#!/bin/bash  
  
read a  
read b  
echo "a=$a b=$b"
```

## Variabili

## Operazioni Matematiche

Solo numeri interi con segno

- Se si usano valori floating non segnala errore ma fa i calcoli con numeri interi

Operazioni ammesse: `+` `-` `*` `/` `%` `<<` `>>` `&` `^` (or esc.)  
`|`

Utilizzo: l'espressione `$(( var1 + var2 ))` restituisce la somma di due variabili

Scrivere `$(( $var1 + $var2 ))` è equivalente

## Variabili

**Esempio:** Si scriva un programma che legge due interi da tastiera e stampa il prodotto

```
#!/bin/bash
read a
read b
c=$(( a * b ))
echo "Il prodotto è $c"
```

### Note:

Osservare la forma `c=$(( a * b ))`

Osservare la concatenazione naturale in `echo "Il prodotto è $c"`

# Strutture di controllo

## Strutture di controllo

### Condizioni

Le condizioni hanno forme

```
if condizione then
    ramo 1
elif condizione2 then
    ramo 2
else
    ramo alternativo
fi
```

Esistono molte sintassi alternative per esprimere le condizioni.  
Ne vedremo una parte.

## Strutture di controllo

### Condizioni tra numeri

Si utilizza la sintassi `(( espressione ))`

Gli operatori di confronto sono i classici: `==` `!=` `<` `>` `<=`

`>=`

Esempio:

```
read n1 n2
if (( n1<n2 ))
then
    echo "$n1 minore di $n2"
elif (( n1==n2 )) then
    echo "$n1 uguale a $n2"
else
    echo "$n1 maggiore di $n2"
fi
```

## Strutture di controllo

### Condizioni tra stringhe

Si utilizza la sintassi `[[ espressione]]`

Gli operatori di confronto sono:

- `=` `!=` : uguaglianza o differenza
- `>` `<` : ordinamento alfabetico
- `-z` : vero se la stringa è vuota (o la variabile non è definita). `! -z` è vero se la variabile non è vuota
- E' necessario usare l'operatore `$` e mettere spazi tra operandi

Esempio: `if [[ $a != $b ]]`

Esempio: `if [[ ! -z $var ]]` : vero se `var` esiste e non è vuota

## Strutture di controllo

## Condizioni tra stringhe

### Esempio:

```
#!/bin/bash
read s1
read s2
if [[ $s1 = $s2 ]]
then
    echo "Le stringhe sono uguali"
else
    echo "Le stringhe sono diverse"
fi
```

## Strutture di controllo

### Condizioni su file

E' molto semplice testare se un file esiste, è vuoto o è una cartella, usando una condizione `if [[ ... ]]` con:

- `-a path` : vero se `path` esiste
- `-f path` : vero se `path` è un file
- `-d path` : vero se `path` è una cartella
- `-s path` : vero se `path` non è vuoto
- `-r path` : vero se posso leggere `path`
- `-w path` : vero se scrivere leggere `path`
- `-x path` : vero se eseguire/attraversare leggere `path`

## Strutture di controllo

**Esempio:** si scriva un programma che legge due path da tastiera. Se sono uguali, controlla che il path corrisponda a una cartella. Se affermativo, stampa il path.

```
#!/bin/bash
echo "Inserisci il primo path:"
read s1
echo "Inserisci il secondo path:"
read s2
if [[ $s1 = $s2 ]]
then
    if [[ -d $s1 ]]
    then
        echo "$s1 è una cartella"
    else
        echo "$s1 non è una cartella"
    fi
else
    echo "Le due stringhe non sono uguali"
fi
```

## Strutture di controllo

### Operatori logici

Si possono creare condizioni composte con gli operatori booleani

- `&&` : and
- `||` : or
- `!` : not

Sintassi: `if condizione1 && condizione2`

Esempio: `if (( a>b )) && [[ $c="hello" ]]`

## Strutture di controllo

### Operatori logici

Si possono usare anche per eseguire comandi secondo una logica voluta

- Ogni comando/programma avviato in bash fornisce alla shell/script chiamante un valore di ritorno
- Per convenzione un comando ritorna: **0** se successo,  $\neq 0$  in caso di errore
- Operatori Booleani usano la *Valutazione a corto circuito*

### Conseguenza:

In bash, il valore **0** è interpretato come `true`

Un valore  $\neq 0$  come `false`

**NOTA:** diverso da altri linguaggio come C o Java!

## Strutture di controllo

## Operatori logici

### Utilizzo in espressioni di comandi

Esegue `comando2` se `comando1` non dà errore

```
comando1 && comando2
```

Esegue `comando2` se `comando1` dà errore

```
comando1 || comando2
```

**Esempio:** eseguo `myprog` solo se la compilazione è andata a buon fine

```
gcc myprog.c -o myprog && ./myprog
```

# Strutture di controllo

## Cicli **for**

### Versione semplice

```
for n in 1 2 3 4
do
    echo "valore di n = $n"
done
```

```
for nome in mario giuseppe vittorio
do
    echo "$nome"
done
```

### Uscita prematura

```
break
```

# Strutture di controllo

## Cicli **for**

### Versione completa

#### Sintassi:

```
for ((INITIALIZATION; TEST; STEP))  
do  
    [COMMANDS]  
done
```

#### Esempio:

```
for ((i = 0 ; i <= 1000 ; i++))  
do  
    echo "Counter: $i"  
done
```

**Nota:** sintassi molto simile al **C**. Importante!

# Strutture di controllo

## Cicli `while`

```
n=0
while ((n<4))
do
    ((n=n+1))
    echo $n
done
```

```
n=0
until((n>4))
do
    ((n=n+1))
done
```

```
while true ; do
    ...
    break
done
```

# Esercizi

## Esercizi

Si scriva uno script bash che ciclicamente chiede all'utente di inserire un file e, se il file esiste, stampa il numero di linee che il file contiene, altrimenti un messaggio di errore. Se l'utente inserisce la stringa `quit`, il programma termina.

```
#!/bin/bash

while true ; do
    echo "Inserisci il nome del file (o 'quit' per uscire): "
    read filename

    if [[ $filename == "quit" ]]; then
        echo "Uscita dal programma."
        break
    fi

    if [[ -f "$filename" ]]; then
        echo "Il file '$filename' ha come numero di linee:"
        wc -l $filename
    else
        echo "Errore: il file '$filename' non esiste."
    fi
done
```

## Esercizi

Si scriva un programma che riceve due argomenti. Se entrambi sono dei file, stampa il contenuto di entrambi

```
#!/bin/bash
if [ "$#" != "2" ]
then
    echo "Servono due argomenti"
else
    if [[ -f $1 ]] && [[ -f $2 ]]
    then
        cat $1
        cat $2
    else
        echo "Non sono due file"
    fi
fi
```

## Esercizi

Si scriva un programma che per ogni file/cartella nella cartella corrente dice se esso è un file o una cartella.

```
#!/bin/bash
for file in *
do
    if [[ -f $file ]]
    then
        echo "$file è un file"
    elif [[ -d $file ]]
    then
        echo "$file è una cartella"
    fi
done
```

## Esercizi

Si scriva un programma che riceve un intero come argomento.  
Se esso è minore di 10, crei i file `0.txt` , ..., `9.txt`

```
#!/bin/bash
if [ "$#" != "1" ]
then
    echo "Serve un argomento"
else
    if (( $1 < 10 ))
    then
        for (( i=0; i<$1 ; i++))
        do
            touch $i.txt
        done
    else
        echo "L'argomento non è minore di 10"
    fi
fi
```

# **Sistemi Operativi**

## **Unità 2: Utilizzo di Linux**

### **Comandi in Bash**

Martino Trevisan  
Università di Trieste  
Dipartimento di Ingegneria e Architettura

## Argomenti

1. Pipe e redirect
2. Filtri e simili
3. Esercizi

# Pipe e Redirect

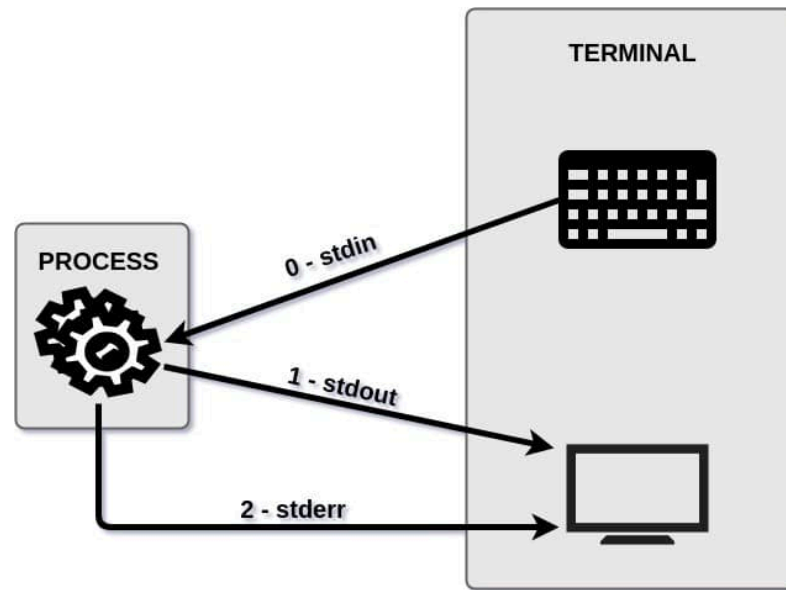
## Pipe e Redirect

In Linux, ogni processo, ha a disposizione 3 canali standard di comunicazione.

- **Standard Input** ( `stdin` ): per ricevere dati in ingresso.
- **Standard Output** ( `stdout` ): per stampare l'output
- **Standard Error** ( `stderr` ): per stampare eventuali errori

## Pipe e Redirect

Di default, un programma riceve lo Standard Input da tastiera, e stampa Standard Output e Standard Error su console.



## Pipe e Redirect

Questo implica quello che già abbiamo visto:

- `read` legge da `stdin`, quindi di default da tastiera
- `echo` stampa su `stdout` che di default è console
- Per stampare su `stderr`, si può usare: `echo "An error!" >&2`. Di default, lo `stderr` è visualizzato a schermo

Tutti i programmi ben scritti, devono attenersi a usare questi *canali standard*.

- Ciò permette una grande flessibilità
- Tutti i programmi di default di Linux lo fanno

## Pipe e Redirect

**Redirezione su file:** è possibile eseguire un programma e redirezionare lo `stdout` su file anzichè stamparlo

**Formato:** `comando > file` oppure `comando 1> file`

Questo perchè `1` indica `stdout` mentre `2` indica `stderr`

**Esempio:**

```
date > data.txt
```

a data corrente viene salvata in `data.txt` e non stampata ad output

**Nota:** se `data.txt` esiste, il contenuto viene sovrascritto

## Pipe e Redirect

***Append su file:*** simile alla redirectione. Il file non viene cancellato, ma lo `stdout` del programma viene aggiunto in coda.

**Formato:** `comando >> file` oppure `comando 1>> file`

**Esempio:**

```
date > file.txt  
sleep 5 # Pausa di 5 secondi  
date >> file.txt # "Appende" a file.txt
```

## Pipe e Redirect

**stderr su file:** permette di redirigere lo `stderr` su un file.

**Formato:** `comando 2> file`

Questo perchè **1** indica `stdout` mentre **2** indica `stderr`

**stdin da file:** permette di prelevare da file anzichè da tastiera lo `stdin` un programma

**Formato:** `comando < file`

## Pipe e Redirect

**Esempio:** Si scriva un programma che riceve due argomenti. Ricerca nella folder corrente tutti i file che hanno il nome del primo argomento e salva la lista nel file il cui nome è il secondo argomento

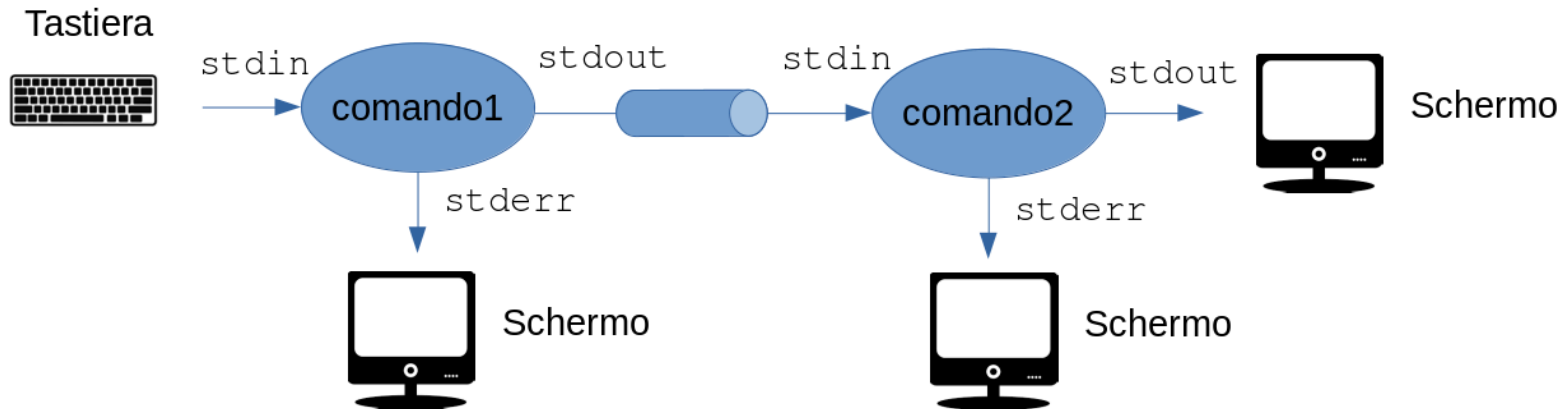
```
#!/bin/bash
if (( "$#" != "2" ))
then
    echo "Servono due argomenti"
else
    find . -name $1 > $2
fi
```

## Pipe e Redirect

**Pipe:** è possibile redirezionare lo `stdout` di un primo comando nello `stdin` di un secondo

**Formato:** `comando1 | comando2`

E' uno dei costrutti più potenti del bash, siccome permette di svolgere compiti complessi con poco codice



Ne vedremo le potenzialità assieme ai comuni comandi bash

## Pipe e Redirect

**Sostituzione:** è possibile usare lo `stdout` di un comando come una variabile.

**Formato:** `$(comando)` oppure ``comando``

**Esempi:**

- `a=$(ls /tmp)` : la stringa `a` contiene l'elenco dei file in `/tmp`
- `rm $( find / -name "*.tmp" )` : rimuove tutti i file nel sistema che terminano per `.tmp`

## Filtri e simili

Negli OS Linux esistono una serie di comandi per manipolare testo.

- Filtrare, ordinare, comporre

Essi si aspettano di lavorare su dati testuali organizzati in righe, come normali file di testo (o di configurazione)

Permettono di fare operazioni complesse con poco codice

- Spesso si usano assieme alle `pipe` al fine di creare pipeline di processamento

# Filtri e simili

## Filtri e simili

`grep [options] pattern [file...]` : stampa le linee del file che contengono il pattern. Se non metto il file usa lo standard input: posso usare grep in pipe. Alcune opzioni:

- `-n` : stampa il numero di riga
- `-i` : case insensitive
- `-c` : stampa il numero di match
- `-v` : stampa solo le linee che **non** contengono il pattern

### Esempio:

- `grep main *.c` : stampa le linee che contengono `main` in tutti i file che finiscono in `.c`
- `ps -ef | grep bash` : stampa tutti i processi che sono istanze del programma `bash`

## Filtri e simili

## Bash Expansion

Osservare il comando `grep main *.c`:

- Esso ricerca il pattern `main` in **tutti** i file che terminano con `*.c`

La **bash** **espande** il termine `*.c` in tutti i file che matchano l'espressione **prima** di eseguire il comando

- `grep` non riceve la stringa `*.c` come argomento
- `grep` riceve già la lista di file

**Esempio:** la cartella corrente contiene `prog.c` e `module.c`

- Il comando `grep main *.c`
- Viene trasformato dalla bash nel comando `grep main module.c prog.c`

## Filtri e simili

# Bash Expansion

Meccanismo flessibile per operare su file:

- Agisce **prima** di avviare il comando
- Il testo non deve essere quotato nè con `*` nè con `'`
- `*` matcha qualsiasi numero di ogni caratteri
- `?` matcha un solo carattere
- `~` rappresenta la home directory:
  - `~/file.txt` equivale a `/home/martino/file.txt` se l'utente è `martino`
  - `.` e `..` non vengono espansi, sono propriamente parte di un path
- Liste racchiuse tra `{ ... }` vengono espanse
  - `mkdir /tmp/{dir1,dir2}` viene espanso in `mkdir /tmp/dir1 /tmp/dir2`

## Filtri e simili

**Esempio:** si scriva un programma che riceve due argomenti: il primo argomento è una cartella, il secondo argomento un pattern. Il programma trova tutte le linee dei file `.c` o `.h` nella cartella, che contengono il pattern. Le linee vengono salvate nel file `/tmp/output.txt`.

```
#!/bin/bash
if (( "$#" != "2" ))
then
    echo "Servono due argomenti"
else
    cat $1/*.c $1/*.h | grep $2 > /tmp/output.txt
fi
```

## Filtri e simili

**cut** : estrarre colonne (o campi) dall'input. Ha diverse modalità.

- **Modalità byte**: estrae i byte specificati da ogni riga. Si utilizza l'opzione `-b byterange`
- **Modalità campi**: estrae i campi specificati, delimitati da un separatore specifico. Si utilizza l'opzione `-d delimitatore -f campi`

## Filtri e simili

**Esempio:** il file `file.txt` contiene:

```
luca 1985 milano  
martino 1990 torino
```

`cat file.txt | cut -b1-2` estrae i primi 2 byte (caratteri) da ogni riga, e stampa su `stdout` :

```
lu  
ma
```

`cat file.txt | cut -d " " -f 2` estrae il secondo campo del file, delimitato da uno spazio. Stampa su `stdout` :

```
1985  
1990
```

## Filtri e simili

`tr [-cds] [set1] [set2]` : legge dei dati e sostituisce i caratteri specificati con altri caratteri. Opzioni comuni:

- `-d` : cancella tutti i caratteri specificati. E' necessario un solo set come argomento
- `-s` : sostituisce le ripetizioni del carattere specificato con un solo carattere

**Esempio:** `tr a A < file1 > file2` : sostituisce le `a` minuscole con `A` maiuscole. Notare lo `stdin` di `tr` è letto da file con l'operatore `<`.

## Filtri e simili

`sort [-dfnru] [-o outfile] [file...]` : Ordina i dati del file o dello `stdin`. Opzioni principali:

- `-f` : tratta maiuscole come minuscole.
- `-n` : riconosce i numeri e li ordina in modo numerico.
- `-r` : ordina i dati in modo inverso.
- `-k` : ordina secondo il numero di colonna dato dopo il k
- `-u` : ordina e rimuove linee duplicate
- `-t SEP` : usa un separatore di campo diverso da quello di default (una *non-blank to blank transition*)

## Filtri e simili

**Esempio:** il file `file.txt` contiene:

```
luca 1985 milano  
martino 1990 torino  
giovanni 1971 trieste
```

`sort < file.txt > sorted.txt` ordina le righe e stampa nel file `sorted.txt`, che conterrà:

```
giovanni 1971 trieste  
luca 1985 milano  
martino 1990 torino
```

`cat file.txt | sort -k 2 -n` ordina le righe per anno (secondo campo) e stampa su `stdout`:

```
giovanni 1971 trieste  
luca 1985 milano  
martino 1990 torino
```

## Filtri e simili

`uniq [-cdu]` : esamina i dati linea per linea cercando linee duplicate e può:

- Di default elimina righe duplicate contigue
- `-c` per ogni riga prepende il numero di occorrenze
- `-d` stampa solo le linee duplicate
- `-u` stampa solo le linee uniche

**Nota:** il comando `uniq` **non** ordina le righe. E' necessario fornirle già ordinate.

`wc [-lwc] [file]` : conta linee ( `l` ), parole( `w` ) e caratteri( `c` ) dello `stdin` o del file

## Filtri e simili

### Utilizzo di `uniq` e `wc`

file.txt

A  
B  
A  
C

cat file.txt | sort | uniq

A  
B  
A  
C

A  
A  
B  
C

A  
B  
C

cat file.txt | sort | uniq -c

A  
B  
A  
C

A  
A  
B  
C

2 A  
1 B  
1 C

cat file.txt | sort | uniq | wc -l

A  
B  
A  
C

A  
A  
B  
C

A  
B  
C

3

## Filtri e simili

### Altri comandi utili (non per esame):

- `sed` : ricerca e sostituzione di espressioni regolati
- `awk` : esecuzione di script (stile C) sulle righe di un file
- `comm` : trova le linee in comune (uguali) tra due file
- `paste` : concatena le linee di più file
- `rev` : scrive l'input in ordine inverso di caratteri, linea per linea

# Esercizi

## Esercizi

Dato il file `vini.txt` contenente il nome, l'anno, la città e il prezzo di alcune bottiglie di vino:

```
ribolla 2012 udine 21
prosecco 2018 trieste 15
barbera 2009 torino 20
freisa 2010 torino 18
barbera 2013 torino 14
barolo 1984 alba 45
```

Si trovino il nome e l'anno del vino più caro:

```
$ sort -k4 -r -n < vini.txt | head -1 | cut -d " " -f 1-2
barolo 1984
```

Si trovino i nomi dei vini prodotti a Torino:

```
$ cat vini.txt | grep torino | cut -d " " -f 1 | sort | uniq
barbera
freisa
```

## Esercizi

Utilizzando lo stesso file dell'esercizio precedente:

Si calcoli quanti vini sono presenti per ogni città:

```
$ cat vini.txt | cut -d " " -f 3 | sort | uniq -c
1 alba
3 torino
1 trieste
1 udine
```

Si calcoli quanti anni passano tra il vino più vecchio e più nuovo:

```
$ min=$(sort -k2 -n < vini.txt | cut -d " " -f 2 | head -n 1)
$ max=$(sort -k2 -n < vini.txt | cut -d " " -f 2 | tail -n 1)
$ echo "Intercorrono $((max-min)) anni"
Intercorrono 34 anni
```

## Esercizi

Dato il file `file.txt` contenente:

```
luca 1985 milano  
martino 1990 torino  
giovanni 1971 trieste  
andrea 1984 milano
```

Si calcoli il numero di righe nel file:

```
wc -l < file.txt # Output 4
```

Si calcoli quante città sono incluse nel file:

```
cat file.txt | cut -d " " -f 3 | sort | uniq | wc -l # Output 3
```

Si trovi la città che appare il maggior numero di volte e il numero di occorrenze:

```
cat file.txt | cut -d " " -f 3 | sort | uniq -c | sort | tail -n 1 # Output 2 milano
```

**Ricorda:** il comando `tail -n N` stampa le ultime `N` righe di un file o dello `stdin`

## Esercizi

Si crei un programma che cerca ricorsivamente tutti i file presenti in una cartella passata come primo argomento.

Collochi quei file in una cartella ricevuta come secondo argomento, suddividendoli in sottocartelle separate per estensione.

**Nota:** si assuma che i nomi di cartelle non contengano `.` e i file ne contengano uno solo, nella forma `nome.estensione`

```
#!/bin/bash

if (( $# != 2 ))
then
    echo "Usage: $0 indir outdir"
    echo "      The program assumes directories do not contain '.'"
    echo "      and files contain one"
    exit 1 # Signal error to the caller
fi

for f in $( find $1 -type f )
do
    folder=$2/$( echo $f | cut -d . -f 2 )
    mkdir -p $folder
    cp $f $folder
done
```